

Track
Collaborate
Experiment
Ship Better
with Git

Git & GitHub

Complete Handbook

From Basics to Real-world workflow

"Better
Developers
Use Version
Control"

① What is Git ?

- Git is a distributed version control system (DVCS) to track changes in your code.
- It helps multiple developers work on the same project without overwriting each other's work.
- It stores the history of your code, so you can go back to any previous version.
- It is fast, lightweight and widely used in the industry.

② Why use Git ?

- ✓ Track changes in your code
- ✓ Collaborate with team members
- ✓ Experiment with new features safely
- ✓ Maintain full history of your project
- ✓ Easily revert to previous versions
- ✓ Work offline (since it's distributed)
- ✓ Industry standard (used in almost every tech company)

③ What is GitHub ?

- GitHub is a cloud platform that hosts Git repositories.
- It provides additional features like
 - Pull Requests
 - Code Reviews
 - Issue Tracking
 - Project Management
 - Collaboration tools
- GitHub is built on top of Git, but Git can be used without GitHub.

Git	GitHub
• Version control system	• Cloud platform
• Runs locally on your machine	• Hosts Git repositories
• Tracks changes	• Collaboration features
• Command line tool	• Web-based (with Git under the hood)
• Does not require internet	• Requires internet



Git powers
GitHub



④ Version Control System (VCS)

- A VCS is a system that records changes to a file or set of files over time.
- It allows you to:
 - Track history
 - Restore previous versions
 - Collaborate with others
 - Manage changes efficiently

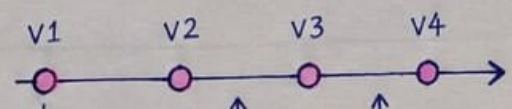
Types of VCS:

- ① Local VCS (e.g. RCS)
- ② Centralized VCS (e.g. SVN)
- ③ Distributed VCS (e.g. Git)

Git is a
Distributed
VCS

⑤ Real-world Analogy

- Think of Git like a 'time machine' for your code.
- You take snapshots (commits) at different points in time.

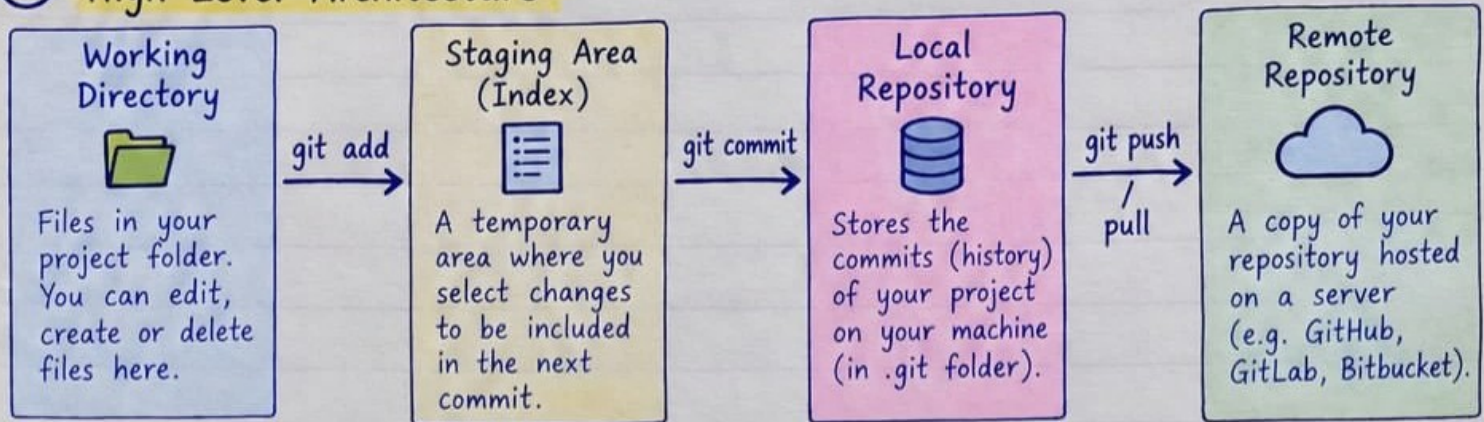


Git Architecture

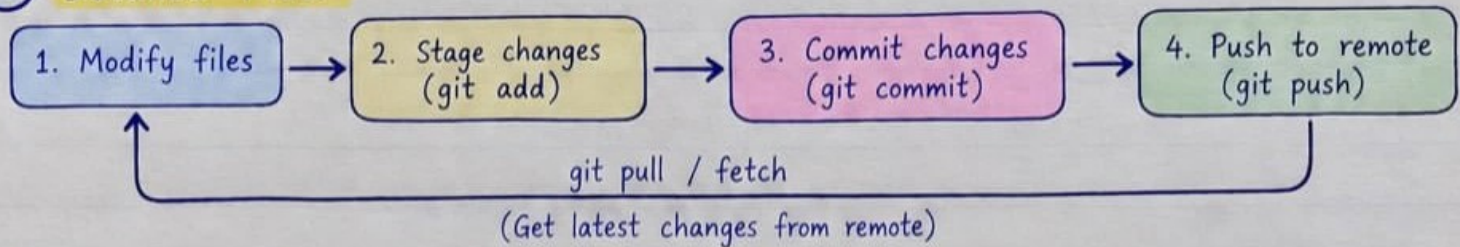
"Git is simple, but powerful."

Understand how Git actually works behind the scenes

① High Level Architecture



② Detailed Flow



③ Inside the .git folder

- When you initialize a Git repo, a hidden folder .git is created.
- It contains all the internal data and history.

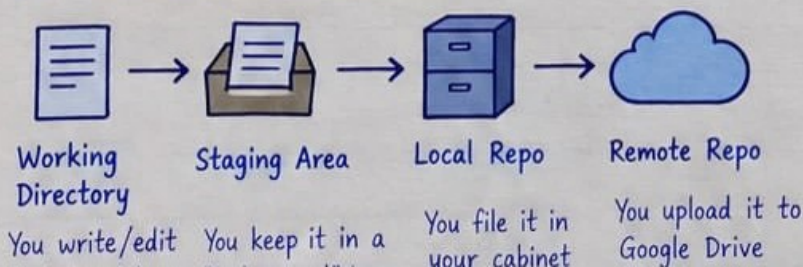
.git/

- HEAD → points to current branch
- objects/ → stores commits, trees, blobs
- refs/ → stores branch and tag references
- config → repository configuration
- index → staging area information
- logs/ → history of reference updates
- hooks/ → scripts (e.g. pre-commit)

④ Key Components

Component	Description
Working Directory	Actual files in your project folder.
Staging Area (Index)	Holds changes you want to commit.
Local Repository	Stores commits and history in .git folder.
Remote Repository	Hosted version of your repo on GitHub/GitLab/Bitbucket.

⑤ Analogy (Real Life)



⑥ Important Points

- ✓ Git is distributed – every developer has a full copy of the repository.
- ✓ You can work offline.
- ✓ Commits are snapshots, not file differences.
- ✓ Each commit has a unique ID (SHA-1/SHA-256).
- ✓ Remote repo is just another Git repository.
- ✓ You can have multiple remotes (e.g. origin, upstream).

Essential Git Commands

Day-to-day commands every developer must know

"Small commands create big changes."

1 Initialize a Repository

- Creates a new Git repository in your project folder.

```
git init
```

This creates a hidden .git folder which stores all Git data.

2 Clone a Repository

- Creates a copy of a remote repository on your local machine.

```
git clone <repo_url>
```

Example:

```
git clone https://github.com/user/repo.git
```

3 Check Status

- Shows the current state of your working directory and staging area.

```
git status
```

Very useful command - tells you what's changed, what's staged and what's not.

4 Add Files to Staging Area

- Adds file changes to the staging area (index).

```
git add <file> # add a specific file
git add . # add all changed files
```

5 Commit Changes

- Records the staged changes in your local repository.

```
git commit -m "Your commit message"
```

Example:

```
git commit -m "Added login feature"
```

💡 Write meaningful commit messages

6 View Commit History

- Shows the list of commits in reverse chronological order (latest first).

```
git log
```

Press 'q' to exit the log view.

Common options:

```
git log --oneline # compact view
git log --graph # show branch graph
git log --all # show all branches
git log -n 5 # last 5 commits
```

7 Show Changes

- Shows the differences between files, commits or working directory.

```
git diff # show unstaged changes
git diff --staged # show staged changes
git diff <commit1> <commit2> # diff between commits
```

8 Show Commit Details

- Shows detailed information about a specific commit.

```
git show <commit_id>
```

Example:

```
git show a1b2c3d
```

9 List Files in a Commit

- Shows the files changed in a specific commit.

```
git show --name-only <commit_id>
```

10 Useful Shortcuts

```
git --version # check git version
git help <command> # get help
git config --list # view git configuration
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```


Git Branching

"Branches keep your ideas safe."

Work on new features without affecting the main codebase

① What is a Branch?

- A branch is a separate line of development in Git.
- It allows you to work on new features, bug fixes, or experiments without affecting the main codebase (e.g. main or master).
- Branches are lightweight and just pointers to commits.

② Why use Branches?

- Work on multiple features in parallel
- Keep the main branch stable
- Easily switch between tasks
- Isolate experimental changes
- Helps in code reviews (via Pull Requests)

💡 Each branch is just a pointer to a commit, not a copy of the entire codebase. That's why branches are fast and lightweight.

③ Common Branch Commands

Command	Description
git branch	List all local branches
git branch -a	List local and remote branches
git branch <name>	Create a new branch
git switch <name>	Switch to an existing branch (modern way)
git checkout <name>	Switch to a branch (older way)
git branch -d <name>	Delete a local branch (after merge)
git branch -D <name>	Force delete a branch (even if not merged)
git rename <old> <new>	Rename a branch

④ Creating and Switching Branches

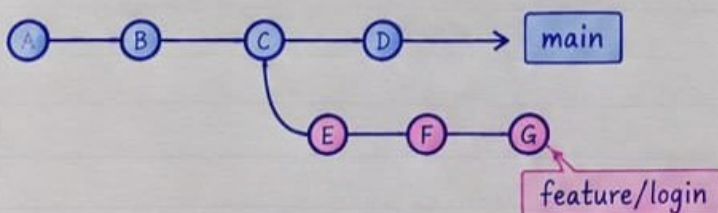
```
# create a new branch
git branch feature/login
# switch to the branch
git switch feature/login
# or (older way)
git checkout feature/login
# create and switch in one command
git switch -c feature/login
```

New in Git 2.23+
→ git switch (recommended over checkout for switching branches)

⑤ List and Delete Branches

```
# list local branches
git branch
# list all branches (local + remote)
git branch -a
# delete a branch (after merge)
git branch -d feature/login
# force delete a branch (not merged)
git branch -D feature/login
```

⑥ Branching Example (Visual)



- = commits on main branch
- = commits on feature branch

Both branches can progress independently. Later, you can merge feature/login back into main.

⑦ Switching Between Branches

```
git switch main      # go to main branch
git switch feature/login # go to feature branch
git switch -         # switch to previous branch
```

⑧ Remote Branches

```
# push a new branch to remote
git push -u origin feature/login
# list remote branches
git branch -r
# delete a remote branch
git push origin --delete feature/login
```


Merging in Git

Combine changes from different branches

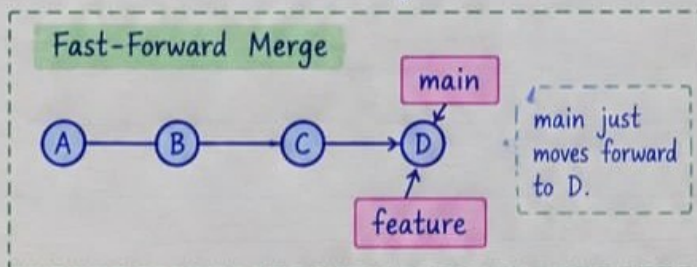
"Merging brings ideas together."

1 What is Merge ?

- Merge combines the changes from one branch into another branch.
- It creates a new commit (in most cases) that brings the histories together.
- It does not delete the source branch.
- It is commonly used to merge a feature branch into main after the feature is complete.

3 Fast-Forward Merge

- Happens when the target branch has no new commits since the feature branch was created.
- Git simply moves the branch pointer forward (no extra merge commit).



5 Merge Commit Message

- Git usually creates a default merge commit message. You can also write a custom message.

```
git merge feature/login
# or with custom message
git merge feature/login -m "Merge login feature"
```

7 Merge Conflicts

- A merge conflict occurs when Git cannot automatically resolve the changes.
- It happens when the same part of a file is modified in both branches.
- You need to manually resolve the conflict.

Conflict Example (in a file)

```
<<<<<< HEAD
console.log("Hello from main");
```

Modify the file, keep the correct

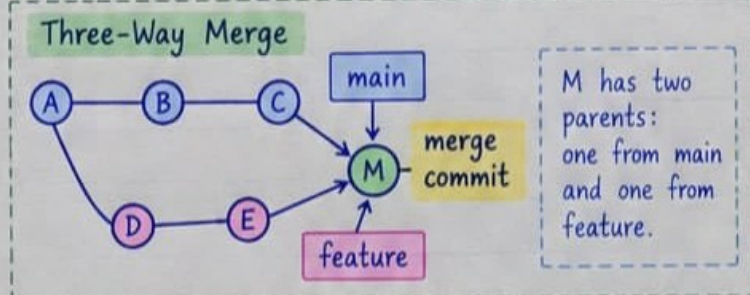
2 Basic Merge Example

```
# switch to the target branch (e.g. main)
git switch main
# merge the feature branch
git merge feature/login
# this will create a merge commit
  (if there are new changes)
```

Always pull latest changes before merging!

4 Three-Way Merge (Normal Merge)

- Happens when both branches have new commits.
- Git creates a new merge commit with two parents (one from each branch).



6 Merge vs Rebase (Quick Comparison)

Feature	Merge	Rebase
History	Keeps full history (non-linear)	Creates linear history
Creates new commit	Yes (merge commit)	No (rewrites history)
Easier to understand	Yes	Can be confusing for beginners
Best use case	Team collaboration, shared branches	Keep history clean (personal branches)

8 Steps to Resolve Merge Conflicts

- 1 Run git status to see conflicted files.
- 2 Open the file and resolve the conflicts manually.
- 3 Remove the conflict markers (<<<<<<, =====, >>>>>>).
- 4 Stage the resolved files: git add <file>
- 5 Complete the merge: git commit



Pro Tips :

Keep your branches small and up to date

Git Rebase

Keep your history clean and linear

"A cleaner history makes a happier team."

① What is Rebase ?

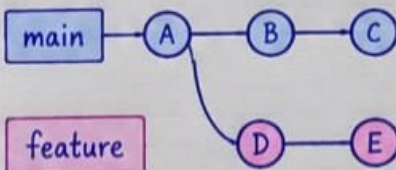
- Rebase moves or reapplies commits from one branch onto another branch.
- It changes the commit history by creating new commits on top of the latest base.
- It helps in keeping a clean, linear history.
- Unlike merge, rebase does not create a merge commit.

② Why use Rebase ?

- ✓ Keeps commit history linear and clean
- ✓ Easy to understand project history
- ✓ Avoids unnecessary merge commits
- ✓ Useful before creating a Pull Request
- ✓ Helps in maintaining a cleaner git log
- ✓ Makes cherry-picking and bisecting easier

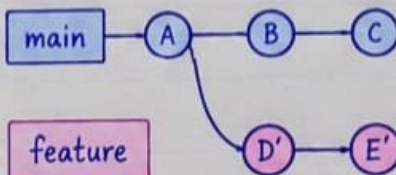
③ How Rebase Works ?

Before Rebase



Feature branch has commits D, E based on an older commit (B).

After Rebase



Commits D and E are reapplied on top of the latest main (C), creating new commits D' and E'.

④ Basic Rebase Command

```
git rebase <base-branch>
```

Example:

```
git rebase main
```

Rebases the current branch onto main

⑤ Interactive Rebase

- Allows you to modify, squash, reorder or delete commits before applying them.

```
git rebase -i <commit>
```

Example:

```
git rebase -i HEAD~3
```

Opens an interactive editor for the last 3 commits.

⑥ Merge vs Rebase

Feature	Merge	Rebase
History	Creates merge commit (non-linear)	Linear history (no merge commit)
Commit IDs	Original commits are preserved	Creates new commit IDs
Best for	Public/shared branches	Feature branches (before merge)
Command	git merge <branch>	git rebase <branch>
Use case	When you want to keep full history	When you want a clean history

⑦ When NOT to use Rebase ?

- ✗ Do not rebase public branches (e.g. main, develop)
- ✗ Avoid rebasing commits that are already shared with others
- ✗ Can cause confusion in team environments
- ✗ Not recommended for released code

⑧ Common Rebase Commands

```

git rebase <branch>      # rebase onto a branch
git rebase -i <commit>   # interactive rebase
git rebase --abort        # cancel the rebase
git rebase --continue     # continue after resolving conflicts
git rebase --skip         # skip the problematic commit
  
```

⑨ Handling Conflicts in Rebase

- ① Run the rebase command.
- ② If there are conflicts, Git will pause.
- ③ Resolve the conflicts in the files.

Same as merge conflicts, but you resolve them



Pro Tip :

- Use rebase to clean up your feature branch before creating a Pull Request.

Undoing Changes in Git

Different ways to undo changes based on your use case

"Mistakes in Git are recoverable."

① Discard Changes in Working Directory

- If you have modified a file but not staged it, you can discard the changes.

```
git restore <file> # discard changes
                    # in a specific file

git restore .       # discard all changes
                    # in current directory
```

③ Undo the Last Commit (Using reset)

- git reset moves the HEAD to a previous commit.
- It can be used in 3 modes: --soft, --mixed, --hard

Mode	What it does	When to use
--soft	Moves HEAD, keeps changes staged	Undo last commit, keep changes
--mixed	Moves HEAD, keeps changes in working dir (un-staged)	Undo last commit, rework changes (default)
--hard	Moves HEAD, discards all changes	Completely undo (commit + changes) (use with caution)

Example:

```
git reset --soft HEAD~1 # undo last commit, keep staged
git reset --mixed HEAD~1 # undo last commit, keep changes
git reset --hard HEAD~1 # undo last commit, discard all
```

⑥ Undo Specific Changes in a File

- If you want to discard changes in specific lines (hunks) interactively.

```
git restore -p <file> # interactive mode
```

② Unstage a File

- If you have added files to staging area (index), but don't want to commit them.

```
git restore --staged <file> # unstage specific file
git restore --staged .      # unstage all files
```

This will keep the changes in your working directory, just removes them from staging area.

④ Revert a Commit

- git revert creates a new commit that undoes the changes of a previous commit.
- It does not modify history (safe for shared branches).

```
git revert <commit-id>
```

Use revert if the commit is already pushed to remote.

Example:

```
git revert a1b2c3d # creates a new commit
                  # that undoes a1b2c3d
```

⑤ Remove Untracked Files

- If you have untracked files (not added to Git) and want to delete them.

```
git clean -f # remove untracked files
git clean -fd # remove untracked files & folders
```

⚠ Be careful! This permanently deletes files.

⑦ Recover Lost Commits

- Even if you mistakenly reset --hard, you can recover commits using reflog.

```
git reflog # see history of HEAD
git reset --hard <commit-id> # recover the commit
```

⑧ Reset vs Revert (Quick Comparison)

Aspect	git reset	git revert
What it does	Moves HEAD to a previous commit	Creates a new commit to undo changes
Affects history	Yes (rewrites history)	No (keeps history)

⑨ Key Takeaways

- ✓ Use git restore to discard working directory changes.
- ✓ Use git reset to undo local commits (choose the right mode).
- ✓ Use git revert for safe undo in shared branches.
- ✓ Use git clean to remove untracked files.
- ✓ Use git reflog to recover lost commits.

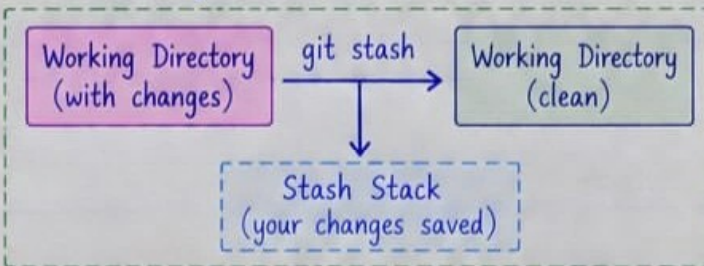
Git Stash & Tags

Temporarily save your work & mark important releases

"Stash the mess today, ship the magic tomorrow."

1 What is Git Stash?

- Git stash temporarily saves your uncommitted changes (both staged and unstaged) and reverts your working directory to a clean state.
- Useful when you want to quickly switch branches or pull latest changes without committing your work.
- Your changes are stored in a stack.



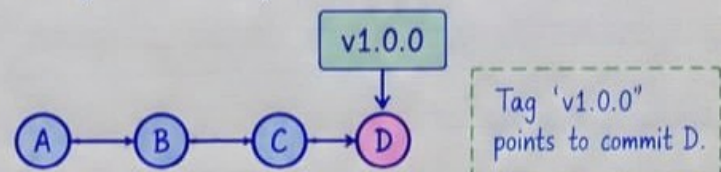
6 Things to Remember (Stash)

- ✓ Stash is local (not pushed to remote).
- ✓ Stashes are stored in a stack (LIFO).
- ✓ Use meaningful messages.
- ✓ git stash pop applies and removes the stash.
- ✓ git stash apply keeps the stash.
- ✓ Use git stash list to view all stashes.

"Stash is your safety net."

7 What are Git Tags?

- Tags are used to mark a specific commit as an important point (usually a release).
- They are like permanent labels for versions.
- Commonly used for release versions (v1.0.0, v2.1.0).
- Tags do not change (unlike branches).



8 Creating Tags

- ```

git tag v1.0.0 # create a lightweight tag
git tag -a v1.0.0 -m "Initial release" # create an annotated tag (recommended)

```

## 9 List, Show and Delete Tags

- ```

git tag # list all tags
git show v1.0.0 # show tag details
git tag -d v1.0.0 # delete a local tag
git push origin --delete v1.0.0 # delete remote tag
  
```

10 Push Tags to Remote

- ```

git push origin v1.0.0 # push a specific tag
git push origin --tags # push all tags

```

## 11 Best Practices (Tags)

- ✓ Use semantic versioning (v1.0.0, v2.1.0).
- ✓ Use annotated tags for releases.
- ✓ Add meaningful messages.

## 2 Basic Stash Commands

- ```

git stash # stash staged + unstaged changes
git stash save "message" # stash with a message (older syntax)
git stash list # list all stashes
git stash show # show latest stash details
git stash show -p # show full diff of latest stash
git stash apply # apply latest stash (keep it)
git stash pop # apply latest stash (and remove)
git stash drop # delete a specific stash
git stash clear # delete all stashes
  
```

3 Stash with Specific File

- ```

git stash push -m "WIP: login feature" src/login.js

```

## 4 Stash Only Staged or Unstaged Changes

- ```

git stash --staged # stash only staged changes
git stash --keep-index # stash unstaged changes (keep staged ones)
  
```

5 Stash Workflow (Example)

- ```

git stash # save current work
git switch main # switch branch
git pull # pull latest changes

```



# GitHub Basics

"More than just code, it's a community."

Host your code, collaborate and build together

## 1 What is GitHub?

- GitHub is a cloud-based platform for hosting Git repositories.
- It allows developers to store, manage, collaborate and track changes in their code.
- Built on top of Git (uses Git under the hood).
- Provides additional features like Pull Requests, Issues, Actions, Projects, Wikis, etc.
- Widely used for open source and private projects.



Git = version control tool

GitHub = platform to host and collaborate using Git

## 2 Git vs GitHub

| Feature        | Git                   | GitHub                                         |
|----------------|-----------------------|------------------------------------------------|
| Type           | Version control tool  | Cloud platform (for Git repositories)          |
| Purpose        | Track changes locally | Host, collaborate and manage code              |
| Works offline? | Yes                   | No (requires internet)                         |
| Developed by   | Linus Torvalds        | Microsoft (acquired in 2018)                   |
| Features       | Commit, branch, merge | Pull Requests, Issues, Actions, Projects, etc. |

## 3 GitHub Account & Repository

- Create a GitHub account at <https://github.com>
- A repository (repo) is a storage space for your project.
- Repos can be public (visible to everyone) or private (only you / selected people).

### Create a New Repository

- 1 Click on "New repository"
- 2 Add a name, description
- 3 Choose public or private
- 4 (Optional) Add README, .gitignore, license
- 5 Click "Create repository"



## 4 Clone a Repository

Clone a remote repository to your local machine:

## 5 Connecting Local Repo to GitHub

If you already have a local Git repo:

```
git remote add origin https://github.com/username/repo.git
git branch -M main
git push -u origin main
```

origin → default name for the remote repository. You can use a different name, but "origin" is common.

## 6 Push, Pull and Fetch

```
git push # upload local changes to remote
git pull # fetch + merge remote changes
git fetch # fetch changes (does not merge)
```

## 7 GitHub Repository Structure

|                            |                   |                 |
|----------------------------|-------------------|-----------------|
| username / awesome-project | ☆ Star 12         | 🍴 Fork 5        |
| <> Code                    | 🔔 Issues          | 🔗 Pull requests |
| ⚙️ Actions                 | ⋮                 |                 |
| 📁 src                      | Initial commit    | 2 days ago      |
| 📁 docs                     | Add documentation | 3 days ago      |
| 📄 README.md                | Update README     | 3 days ago      |
| 📄 .gitignore               | Add .gitignore    | 3 days ago      |
| 📄 package.json             | Initial commit    | 3 days ago      |

## 8 Other Useful GitHub Features

| Feature       | Use                                 |
|---------------|-------------------------------------|
| Issues        | Track bugs, tasks, feature requests |
| Pull Requests | Propose and review code changes     |
| Actions       | Automate workflows (CI/CD)          |
| Projects      | Manage tasks with boards            |
| Wiki          | Document your project               |
| Discussions   | Have conversations with community   |
| Releases      | Publish stable versions (with tags) |
| Stars         | Show appreciation (like)            |
| Fork          | Create your own copy of a repo      |

## 9 Pro Tips



- ✓ Write a good README.md
- ✓ Use .gitignore to exclude unnecessary files
- ✓ Keep commit messages meaningful
- ✓ Use issues to track work



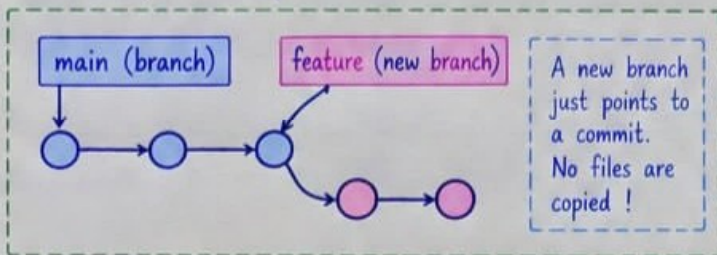
# Git Branching

Work in parallel, merge with confidence

"Branches turn ideas into reality."

## 1 What is a Branch?

- A branch is a lightweight, movable pointer to a commit.
- It allows you to work on new features, fixes or experiments without affecting the main code.
- Each branch has its own commit history, but all branches share the same codebase.
- Branches make collaboration and parallel development easy.



## 2 Useful Branch Commands

|                        |                                |
|------------------------|--------------------------------|
| git branch             | # list all local branches      |
| git branch -a          | # list local + remote branches |
| git branch <name>      | # create a new branch          |
| git checkout <name>    | # switch to a branch (older)   |
| git switch <name>      | # switch to a branch (newer)   |
| git checkout -b <name> | # create and switch            |
| git branch -d <name>   | # delete a branch (safe)       |
| git branch -D <name>   | # force delete a branch        |
| git rename <old> <new> | # rename a branch              |

## 3 Creating and Switching Branches

```
create a new branch and switch
git checkout -b feature/login ← (older way)
or using switch (recommended)
git switch -c feature/login ← (newer way)
check current branch
git branch --show-current
```

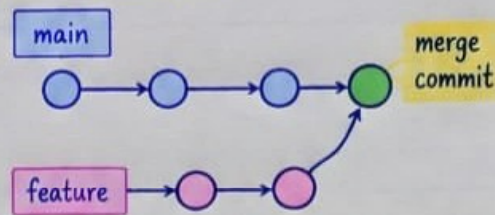
## 4 Working with Remote Branches

|                             |                         |
|-----------------------------|-------------------------|
| git push -u origin <branch> | # push and set upstream |
| git fetch                   | # fetch latest changes  |
| git pull                    | # fetch + merge         |

## 5 Merging Branches

- After completing work in a feature branch, you merge it back into the main (or target) branch.
- Use git merge to combine changes.
- Always test your code before merging.

### Merge Example



Merging combines the histories of both branches and creates a new commit.

## 6 Merge Strategies

| Strategy         | Description                                    | When to use                              |
|------------------|------------------------------------------------|------------------------------------------|
| Fast-forward     | Moves branch pointer forward (no merge commit) | When no new commits in target            |
| Three-way merge  | Creates a new merge commit                     | Most common case                         |
| Squash merge     | Combines all commits into one                  | Keep clean history (e.g. GitHub)         |
| Rebase and merge | Replays commits on top of target               | Linear history (preferred in some teams) |

## 7 Rebasing (Alternative to Merge)

- Rebase moves your commits on top of the latest target branch.
- It gives a clean, linear history (no merge commits).
- Be careful when rebasing shared branches.

```
rebase feature branch onto latest main
git checkout feature/login
git rebase main
```

Use rebase for a cleaner history, especially in personal branches.

## 8 Best Practices

- ✓ Use meaningful branch names (e.g. feature/login).
- ✓ Keep branches small and focused.
- ✓ Pull latest changes before starting work.
- ✓ Use pull requests for code review.
- ✓ Delete merged branches to keep the repo clean.
- ✓ Avoid direct commits to main.



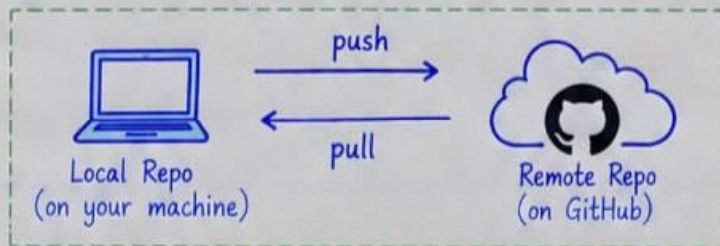
# Remote Repositories

Work beyond your local machine

"Push your progress, not perfection."

## 1 What is a Remote Repository ?

- A remote repository is a version of your project that is hosted on a server (e.g. GitHub, GitLab, Bitbucket).
- It allows multiple people to collaborate on the same project.
- You can push your local changes to the remote repo and pull changes made by others.
- Your remote repo can be public or private.



## 5 Pulling Changes from Remote

- Use `git pull` to fetch and merge changes from the remote repository to your local branch.

```
git pull origin <branch-name>
```

Example:

```
git pull origin main
```

- This downloads the latest changes and merges them into your current branch.
- Always pull before starting new work to avoid conflicts.

## 6 Cloning a Repository

- If you want to work on an existing project, you can clone it to your local machine.

```
git clone https://github.com/username/repo.git
```

Example:

```
git clone https://github.com/abhi_techhub/myapp.git
```

- This creates a local copy of the remote repository with full history.
- After cloning, a remote named `origin` is automatically set.

## 7 Common Remote Commands

| Command                                                | Description                    |
|--------------------------------------------------------|--------------------------------|
| <code>git remote -v</code>                             | List remote repositories       |
| <code>git remote add &lt;name&gt; &lt;url&gt;</code>   | Add a new remote               |
| <code>git remote remove &lt;name&gt;</code>            | Remove a remote                |
| <code>git remote rename &lt;old&gt; &lt;new&gt;</code> | Rename a remote                |
| <code>git fetch origin</code>                          | Fetch changes (does not merge) |
| <code>git pull origin &lt;branch&gt;</code>            | Fetch + merge changes          |
| <code>git push origin &lt;branch&gt;</code>            | Push changes to remote         |
| <code>git clone &lt;url&gt;</code>                     | Clone a remote repository      |

## 8 Best Practices

- ✓ Use meaningful repository names.
- ✓ Keep your remote repository updated.
- ✓ Pull before pushing to avoid conflicts.
- ✓ Use `.gitignore` to avoid pushing unnecessary files.
- ✓ Use feature branches for new work.
- ✓ Keep your code and commit messages clean.

## 2 Adding a Remote Repository

Syntax:

```
git remote add origin <repo-url>
```

Example:

```
git remote add origin https://github.com/username/repo.git
```

- `origin` is a common name for the remote repository.
- You can use a different name, but `origin` is the convention.

## 3 View Remote Repositories

```
git remote -v
```

# shows remote URLs

Example Output:

```
origin https://github.com/username/repo.git (fetch)
origin https://github.com/username/repo.git (push)
```

## 4 Pushing Changes to Remote

- 1 Make sure you have committed your changes.
- 2 Use `git push` to upload your commits to remote.

```
git push origin <branch-name>
```

Example:

```
git push origin main
```



# Git Workflows

Different ways teams use Git in real projects

"Good workflow turns individual effort into team success."

## 1 Why Git Workflows ?

- In real projects, multiple developers work on different features at the same time.
- Workflows help in managing branches, code reviews and releases in a structured way.
- They reduce conflicts and make collaboration easier.
- There is no single "best" workflow — it depends on your team size, project and release process.

"A good Git workflow keeps your team aligned and your codebase healthy."

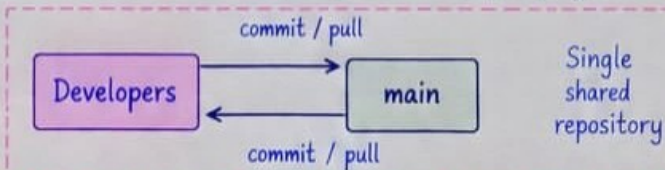
## 2 Types of Git Workflows

- Common workflows used in industry:

1. Centralized Workflow
2. Feature Branch Workflow (most common)
3. Git Flow (for large projects)
4. Forking Workflow (for open source)
5. Trunk Based Development (modern approach)

## 3 Centralized Workflow (Simple)

- All developers work on a single main branch.
- Suitable for small teams and simple projects.



### Pros:

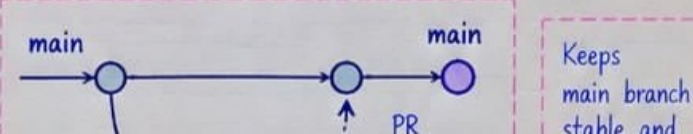
- Simple to understand
- Easy to maintain

### Cons:

- Not suitable for large teams
- High chance of conflicts

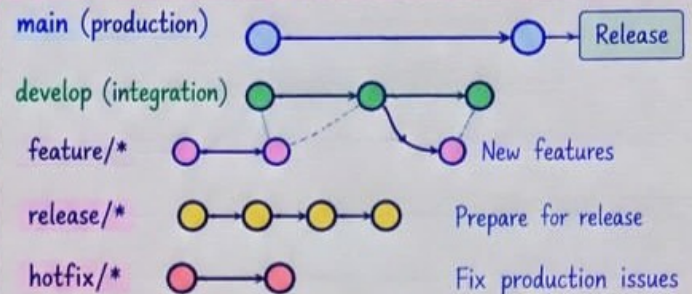
## 4 Feature Branch Workflow (Most Common)

- Each feature/bug fix gets its own branch.
- Developers create a branch, make changes and raise a Pull Request (PR) for review.
- After approval, it is merged into main (or develop).



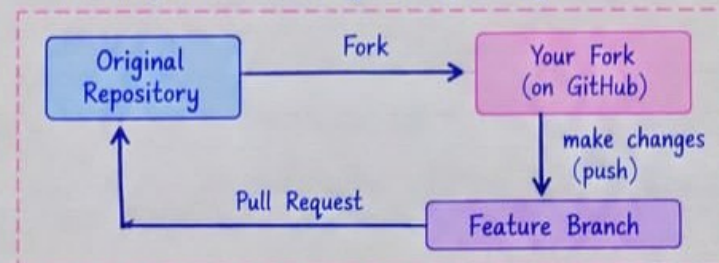
## 5 Git Flow (For Large Projects)

- Uses multiple long-lived branches.
- Good for projects with scheduled releases (e.g. enterprise applications).



## 6 Forking Workflow (Open Source)

- Each contributor forks the repository to their own GitHub account.
- They make changes, push to their fork and raise a Pull Request to the original repository.



## 7 Trunk Based Development (Modern)

- Developers work on short-lived branches and merge frequently into main (trunk).
- Used in modern CI/CD environments with automated testing.
- Keeps the codebase up to date and reduces long-lived branches.

Requires good testing and CI/CD setup.

## 8 Which Workflow to Choose ?

| Workflow       | Best for             | Key Benefit                     |
|----------------|----------------------|---------------------------------|
| Centralized    | Small teams          | Simple                          |
| Feature Branch | Most projects        | Keeps main stable               |
| Git Flow       | Large / Enterprise   | Structured releases             |
| Forking        | Open source          | External contributions          |
| Trunk Based    | Modern teams (CI/CD) | Fast and continuous integration |

## 9 Key Takeaways

- ✓ Choose a workflow based on team size and project needs.
- ✓ Use branches to keep your code organized.



# Resolving Conflicts & Advanced Git

"Conflicts are just conversations between changes."

Conflicts happen, but Git gives you the power to resolve them.

## 1 Why Conflicts Happen ?

- When two or more people modify the same part of a file and try to merge their changes.
- Git doesn't know which change is correct, so it asks you to resolve the conflict.
- Conflicts are common in team environments and are a normal part of development.

"A conflict means Git needs your help, not that something is broken."

## 2 Example of a Conflict

Suppose both you and a teammate modified the same line in app.js

```
// Your changes (HEAD)
const port = 3000;
```

```
// Teammate's changes
const port = 4000;
```

After merging, the file will look like:

```
<<<<<< HEAD
const port = 3000;
=====
const port = 4000;
>>>>>> feature/login
```

Everything between <<<<<< and >>>>>> is a conflict block. You need to manually edit and keep the correct code.

## 3 Steps to Resolve a Conflict

- 1 Open the conflicted file.
- 2 Identify the correct changes (or combine both).
- 3 Edit the file and remove the conflict markers  
<<<<<<, =====, >>>>>>.
- 4 Save the file.
- 5 Stage the resolved file using `git add <file>`.
- 6 Complete the merge using `git commit`.

## 4 Example Resolution

```
// Final resolved code
const port = 3000; // keep your change
```

```
git add app.js
git commit -m "Resolve merge conflict in app.js"
```

## 5 Useful Git Commands (Advanced)

| Command                                      | Description                                   |
|----------------------------------------------|-----------------------------------------------|
| <code>git diff</code>                        | Show changes between commits, branches, files |
| <code>git diff --staged</code>               | Show changes in staging area                  |
| <code>git log --oneline --graph</code>       | Compact commit history with branch graph      |
| <code>git reflog</code>                      | Show history of HEAD (useful for recovery)    |
| <code>git cherry-pick &lt;commit&gt;</code>  | Apply a specific commit to current branch     |
| <code>git rebase &lt;branch&gt;</code>       | Reapply your commits on top of another branch |
| <code>git stash</code>                       | Temporarily save uncommitted changes          |
| <code>git stash pop</code>                   | Apply the latest stashed changes              |
| <code>git reset --soft &lt;commit&gt;</code> | Move HEAD but keep changes staged             |
| <code>git reset --hard &lt;commit&gt;</code> | Discard all changes and move HEAD             |
| <code>git stash -fd</code>                   | Remove untracked files and directories        |

## 6 Rebase vs Merge

| Rebase                            | Merge                          |
|-----------------------------------|--------------------------------|
| Creates a linear history          | Creates a merge commit         |
| Cleaner commit history            | Preserves branch history       |
| Better for small feature branches | Better for long-lived branches |
| Rewrites commit history           | Does not rewrite history       |
| Can be risky on shared branches   | Safer for team collaboration   |

## 7 Stash (When you are not ready to commit)

- Use `git stash` to temporarily save your work.
- Useful when you want to quickly switch branches without committing.

```
git stash
git stash list
git stash pop
git stash apply
```

Stash is like a "clipboard" for your changes.

## 8 Recovering Lost Work

- Use `git reflog` to see the history of where your HEAD has been.
- You can recover lost commits, branches or even deleted files.

```
git reflog
git checkout <commit-hash> # to recover
```



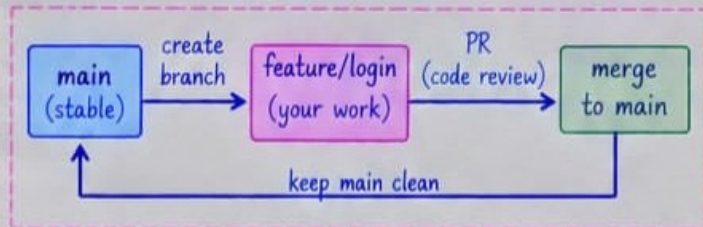
# Git in Real Projects

Practical Tips, Team Workflow, .gitignore, Hooks, and More

"It's not just Git commands, it's a better way to work."

## 1 Typical Team Workflow

- Most teams follow a branch-based workflow (e.g. Feature Branch or Git Flow).
- Each developer works on a separate branch.
- Code is reviewed using Pull Requests (PR).
- Changes are merged after approvals and CI checks.



## 2 Common Branch Naming Conventions

| Type       | Branch Name       | Example             |
|------------|-------------------|---------------------|
| Feature    | feature/<name>    | feature/login       |
| Bug Fix    | fix/<issue>       | fix/header-bug      |
| Hotfix     | hotfix/<issue>    | hotfix/payment-fail |
| Release    | release/<version> | release/v1.2.0      |
| Experiment | exp/<name>        | exp/new-ui          |

Use meaningful names so that everyone understands the purpose of the branch.

## 3 Writing Good Commit Messages

- Use clear, concise and meaningful messages.
- Follow a consistent format (e.g. Conventional Commits).
- A good commit message should explain "what" and "why".

```

feat: add login functionality # new feature
fix: resolve header alignment issue # bug fix
docs: update README # documentation
refactor: simplify API service # code improvement

```

## 4 Using .gitignore

- .gitignore tells Git which files/folders to ignore.
- Important to avoid pushing unnecessary or sensitive files.

Example .gitignore

```

node_modules/
.env
.DS_Store
lib/

```

Keep your repository clean and secure.

## 5 Handling Large Files

- Avoid committing large files (builds, videos, datasets).
- Use Git LFS (Large File Storage) for large binaries.
- Or store them in external storage (e.g. S3, Drive).

# track a file with Git LFS

```

git lfs install
git lfs track "*.zip"
git add .gitattributes
git commit -m "Add LFS tracking"

```

Regular Git is not ideal for files > 100MB.

## 6 Useful .gitattributes

- Use .gitattributes to handle line endings and binary files.

# Auto handle line endings

```
* text=auto
```

# Treat images as binary

```
*.png binary
```

```
*.jpg binary
```

```
*.pdf binary
```

Helps avoid merge issues across different OS (Windows, Mac, Linux).

## 7 Git Hooks (Automation)

- Git hooks are scripts that run automatically on certain events (e.g. before commit, before push).
- Useful for running linters, tests or validation.

Example: pre-commit hook (using Husky)

```

npx husky install
npx husky add .husky/pre-commit "npm test"
git add .husky/pre-commit
git commit -m "Add pre-commit hook"

```

Helps maintain code quality in the team.

## 8 Real Project Tips

- ✓ Keep main branch stable and always deployable.
- ✓ Use feature branches for new work.
- ✓ Take regular pulls from main to avoid conflicts.
- ✓ Write meaningful commit messages.
- ✓ Use PRs for code reviews.
- ✓ Keep your .gitignore and .gitattributes updated.
- ✓ Use Git hooks and CI/CD for quality checks.
- ✓ Communicate with your team before big changes.
- ✓ Clean up old branches after merge.



# Best Practices, Common Mistakes & Conclusion

"Small Good Practices make a Big Difference."

Write better Git. Build better projects. Grow as a developer.

## 1 Git Best Practices

- ✓ Use meaningful and consistent branch names.
- ✓ Make small, focused commits.
- ✓ Write clear commit messages.
- ✓ Pull latest changes before starting new work.
- ✓ Use .gitignore to avoid unnecessary files.
- ✓ Keep your main branch stable and deployable.
- ✓ Use feature branches and Pull Requests.
- ✓ Review code before merging.
- ✓ Use tags/releases for versions.
- ✓ Regularly clean up old branches.
- ✓ Use Git hooks and CI/CD for automation.
- ✓ Keep sensitive data out of the repository.
- ✓ Follow your team's workflow (e.g. Git Flow).
- ✓ Document important setup steps in README.

## 2 Common Mistakes to Avoid

- ✗ Committing directly to main branch.
- ✗ Using unclear commit messages like "fixed".
- ✗ Not pulling latest changes (leads to conflicts).
- ✗ Pushing large/binary files (e.g. videos, datasets).
- ✗ Forgetting to add .gitignore.
- ✗ Accidentally committing sensitive data (API keys).
- ✗ Creating too many unnecessary branches.
- ✗ Not reviewing code before merge.
- ✗ Force pushing on shared branches.
- ✗ Not resolving conflicts properly.
- ✗ Ignoring CI/CD failures.
- ✗ Keeping old or merged branches indefinitely.

## 3 Quick Git Cheat Sheet

|                         |                          |
|-------------------------|--------------------------|
| git init                | # initialize a new repo  |
| git clone <url>         | # clone a repository     |
| git status              | # check current status   |
| git add .               | # stage all changes      |
| git commit -m "message" | # commit changes         |
| git push                | # push to remote         |
| git pull                | # pull latest changes    |
| git branch              | # list branches          |
| git checkout -b <name>  | # create & switch branch |
| git merge <branch>      | # merge branch           |
| git log --oneline       | # view commit history    |
| git diff                | # see differences        |
| git stash               | # stash changes          |

## 4 Git vs Other Version Control Systems

| Feature      | Git                  | SVN         | Mercurial   |
|--------------|----------------------|-------------|-------------|
| Type         | Distributed          | Centralized | Distributed |
| Speed        | Fast                 | Slower      | Fast        |
| Offline Work | Yes                  | No          | Yes         |
| Branching    | Lightweight          | Heavy       | Good        |
| Popularity   | Very High            | Moderate    | Low         |
| Used by      | GitHub, GitLab, etc. | Enterprises | Some orgs   |

Git is the most popular VCS because of its speed, flexibility and strong community support.

## 5 Real-World Tools & Platforms

- GitHub – most popular hosting platform.
- GitLab – open source, self-hostable.
- Bitbucket – popular in enterprise teams.
- Azure DevOps – integrates with Microsoft tools.
- Git (CLI) – core tool for version control.
- GitKraken / SourceTree – GUI tools (optional).

## 6 Key Takeaways

- ✓ Git helps teams work in parallel without conflicts.
- ✓ Good branching and commit practices make collaboration smooth.
- ✓ Always review and test your code before merging.
- ✓ Use tools like .gitignore, hooks and CI/CD to improve productivity.
- ✓ Git is not just a tool, it's a way of working.
- ✓ Practice regularly – the more you use Git, the more comfortable you'll become!

## 7 Next Steps (Keep Learning)

- ☐ Practice Git commands in a personal project.
- ☐ Try GitHub Pull Requests and code reviews.
- ☐ Explore advanced topics like rebasing, cherry-picking.
- ☐ Set up Git hooks in one of your projects.
- ☐ Learn about CI/CD integration (GitHub Actions, GitLab CI).
- ☐ Contribute to an open source project.
- ☐ Keep exploring and building!